

Übung 3

13. April 2026

Übersicht

Hinweis

Dieses Übungsblatt ist eine zusammenhängende Aufgabe. Lesen Sie zunächst das gesamte Übungsblatt sorgfältig und verschaffen Sie sich einen Überblick über die Anforderungen. Analysieren Sie anschließend die bereitgestellte Projektvorlage, bevor Sie mit der eigentlichen Implementierung beginnen.

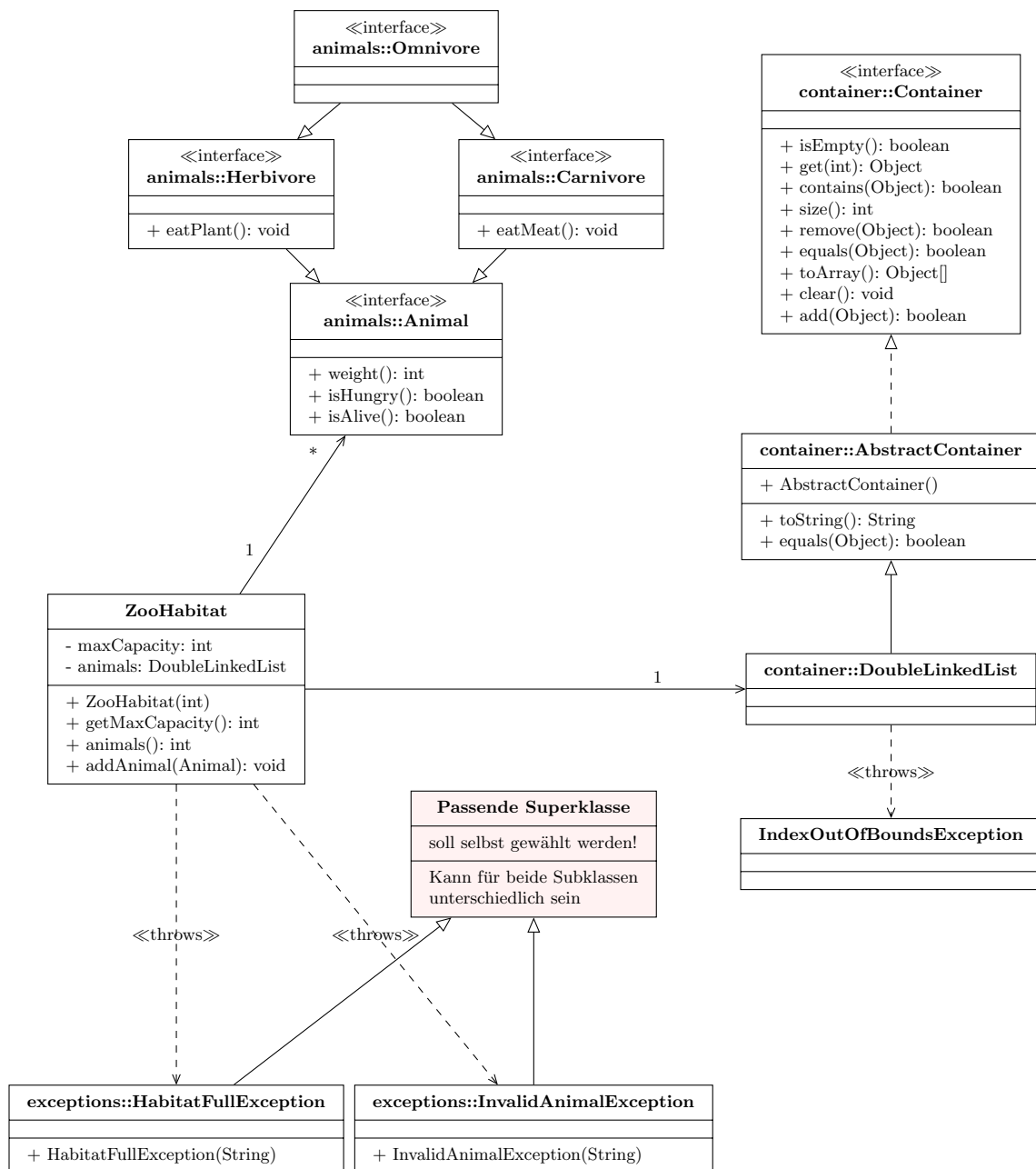


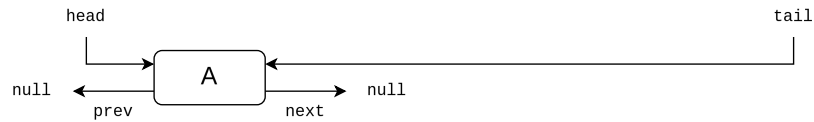
Abbildung 1: Unvollständiges Klassendiagramm

Aufgabe 01: DoubleLinkedList

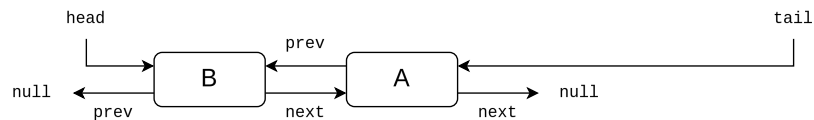
In dieser Aufgabe sollen Sie eine eigene Implementierung einer **doppelt verketteten Liste** erstellen. Diese Datenstruktur speichert Objekte so, dass jedes Element sowohl einen Verweis auf den **Vorgänger** als auch auf den **Nachfolger** enthält.

Zur Veranschaulichung dient die folgende grafische Darstellung:

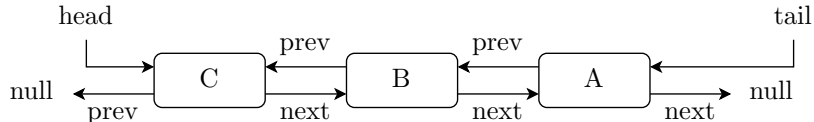
1. Einfügen von ListElement A



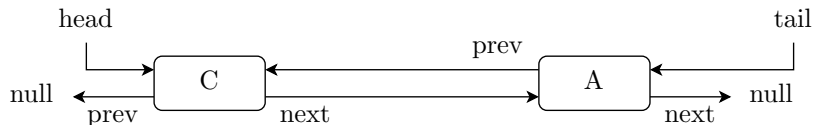
2. Einfügen von ListElement B



3. Einfügen von ListElement C



4. Löschen von ListElement B



- Implementieren Sie die Klasse `DoubleLinkedList` im Paket `de.tha.prog2.container`.
- Die Klasse soll von `AbstractContainer` erben und dadurch das Interface `Container` implementieren.
- Zur Verwaltung der Elemente soll eine Hilfsklasse `ListElement` verwendet werden.
- Verwalten Sie zwei Referenzen:
 - `head` – zeigt auf das erste Element
 - `tail` – zeigt auf das letzte Element

Die Methoden, die **nicht in `AbstractContainer` oder als default im Interface `Container` implementiert sind**, müssen in `DoubleLinkedList` konkret implementiert werden. Dazu gehören unter anderem:

- `Object get(int index)`
Gibt das Objekt am angegebenen Index zurück.
Bei ungültigem Index ist eine `IndexOutOfBoundsException` zu werfen.
- `boolean remove(Object o)`
Entfernt das erste Vorkommen eines Objekts (nach `equals()`).

Testen

Schreiben Sie eine eigene Klasse `MyContainerTest`, um Ihre Implementierung der `DoubleLinkedList` zu testen. Wenn Sie der Meinung sind, dass Ihre Implementierung korrekt funktioniert, können Sie den UnitTest `DoubleLinkedListTest` verwenden, um die Funktionalität zu überprüfen.

Aufgabe 02: ZooHabitat**(a) Das Rahmenprogramm**

Im Klassendiagramm in Abbildung 1 sehen Sie eine unvollständige Übersicht der Interfaces und Klassen, die für dieses Aufgabenblatt relevant sind. Erstellen Sie alle notwendigen Klassen, Exceptions und Interfaces in den passenden Paketen `de.tha.prog2`, `de.tha.prog2.exceptions` und `de.tha.prog2.animals`. Bereits vorhandene Klassen müssen Sie selbstverständlich nicht erneut anlegen.

Erstellen Sie danach zunächst sogenannte **Dummy-Implementierungen**: Dabei werden alle im Diagramm angegebenen Methoden definiert, jedoch noch ohne sinnvolle Logik versehen. Ziel dieser Aufgabe ist es, ein kompilierbares Klassensystem zu erstellen. Die eigentliche Funktionalität wird im weiteren Verlauf ergänzt.

Hinweis

Das Klassendiagramm ist bewusst unvollständig. Sollten Sie zusätzliche Attribute oder Methoden benötigen, fügen Sie diese eigenständig hinzu.

Legen Sie außerdem eigene Tierklassen an, die die Interfaces **Carnivore** (Fleischfresser), **Herbivore** (Pflanzenfresser) und **Omnivore** (Allesfresser) implementieren. Welche konkreten Tierarten Sie dafür verwenden, können Sie frei wählen.

(b) Interfaces und Exceptions

Nachdem das Rahmenprogramm steht, soll nun die eigentliche Funktionalität der verbleibenden Klassen umgesetzt werden.

Erstellen Sie dazu eine Testklasse **MyZooTest**, mit der Sie Ihr Programm systematisch prüfen können, während Sie das Rahmenprogramm weiter entwickeln. Überlegen Sie dazu selbstständig, welche Funktionalitäten besonders wichtig sind, und entwickeln Sie passende Testmethoden.

Die folgenden Anforderungen sollen durch Ihre Implementierung erfüllt werden:

- Ein Zoo besteht aus mehreren Gehegen, in denen Tiere leben. Diese Gehege werden durch die Klasse **ZooHabitat** modelliert. Es ist nicht erforderlich, einen vollständigen Zoo zu implementieren. Beschränken Sie sich stattdessen auf die Klasse **ZooHabitat** und verwenden Sie für Ihre Tests mehrere Instanzen (z. B. zwei Gehege).
- Jedes Gehege besitzt eine maximale Kapazität an Tieren. Wird versucht, über die Methode **addAnimal()** ein weiteres Tier in ein bereits volles **ZooHabitat** einzufügen, soll eine **HabitatFullException** ausgelöst werden. Beachten Sie auch, dass ein Tier nicht zweimal hinzugefügt werden kann.
- Tiere dürfen innerhalb eines Geheges nicht gemischt werden: **Herbivore** (Pflanzenfresser), **Carnivore** (Fleischfresser) und **Omnivore** (Allesfresser) müssen in getrennten Gehegen gehalten werden. Versucht man, ein Tier hinzuzufügen, das nicht dem Typ der bereits im Gehege lebenden Tiere entspricht, soll eine **InvalidAnimalException** geworfen werden.
- Die Methode **animals()** soll alle Tiere eines Geheges auflisten und zusätzlich die aktuelle Anzahl der Bewohner zurückgeben. Die Ausgabe sollte informativ und gut lesbar sein, idealerweise mit Tierart, ggf. Namen, Alter und Gewicht der Tiere.

Beispielausgabe

```
Im Habitat leben folgende Fleischfresser:
- Shir Khan, Tiger, 4 Jahre, 150 kg
- Woods, Tiger, 14 Jahre, 200 kg
```

Testen

Nachdem Sie Ihre Implementierung mit Hilfe Ihrer Klasse **MyZooTest** getestet haben, können Sie den UnitTest **ZooHabitatTest** verwenden, um die Funktionalität zu überprüfen.